Building a Stable AI Development Environment

AI Coding Agent Workshop — Intermediate Session (Claude)

Goal: Understand how Prompt, Context, and Harness work together so AI can work more consistently across a team

John Wu | Apr 2026

Today's Journey

Segment	What we'll do
Concepts	See Prompt / Context / Harness through a restaurant lens
Live Demo	Watch the same request become stable step by step
Exercise Time	Apply it to your own work scenario
Recap & Q&A	Key takeaways + take-home practice

> The goal isn't to memorize more terms — it's to learn how to fit AI into a repeatable workflow

Part 1: Concepts

The "Opening a Restaurant" analogy

What you say	Restaurant analogy	Concept
"I want something spicy"	Cook a first draft and iterate	**Vibe Coding**
"Kung Pao chicken, no peanuts, less oil"	The more specific, the better the dish	**Prompts**
"This is a Sichuan restaurant, high-heat wok"	Kitchen rules are written down	**Project Instructions**
Recipe + checkpoints + tool list	Standard operating procedure	**Skill**
Connect to suppliers, warehouse, delivery	External interfaces	**MCP**

What Are We Upgrading Today?

Angle	What you're doing
Single collaboration	State the request clearly, let AI complete one task
Repeatable workflow	Design rules, flows, tools so AI can handle many tasks reliably

> You're not just cooking one dish — you're designing how the whole kitchen runs
> The same dish: some cooks rely on luck every time, others rely on systems for consistent output

Today's Tool Perspective

Tool	Role today
Claude Code	Primary demo tool — shows how to design the AI work environment
Codex	Occasional comparison — confirms the concepts are tool-agnostic

The point isn't comparing tools — it's moving the principles

How do these concepts transfer to the tools you already use?

> Tool names change, but Prompt / Context / Harness as a mindset holds up

The Three Pillars of an AI Work Environment

Pillar	Restaurant metaphor	In plain words
Prompt	How you order	The instructions, constraints, and output you give AI
Context	Menu, kitchen rules, ingredient info	The project knowledge and external info AI can reach
Harness	Kitchen rules, permissions, safety checks	What AI is allowed to do, how far, and when to stop

> Intermediate isn't learning more jargon — it's knowing which layer is undercooked

Why Does the Same Request Still Drift?

Usually it's more than one reason

1. **Prompt isn't clear enough** — task, constraints, and output format aren't spelled out
2. **Context is incomplete** — AI doesn't know the repo conventions, existing code, or external data
3. **Harness isn't solid** — no checkpoints, permission boundaries, or workflow constraints

What you need to learn isn't "write a giant prompt"

It's judging: which layer needs fixing this time?

> The problem isn't always your asking — the kitchen might just not know

Prompt — How You Order

Not every task needs the same level of tightness

Task type	Ordering style	Suggested tightness
Database migration	Specify the approach, limit the risk	**Strict**
API implementation	Clear spec, some room to maneuver	**Medium**
Code Review	Give the goal and what to look for	**Medium to loose**
Writing docs	State the audience and format	**Loose**

> The more dangerous the action, the more explicit the prompt — exploratory tasks can breathe more

Context — What the Kitchen Knows

If Prompt is ordering, then Context is:

- What kind of restaurant this is
- What rules the kitchen follows
- What ingredients are on hand right now
- Whether there are external systems to pull data from

```
typical context = conversation + relevant files + project instruction files + external info fetched on demand
⚠️ context affects not only quality but also cost and efficiency — stuffing irrelevant things degrades judgment and wastes resources
```

> AI doesn't only hear what you say in the moment — it's also shaped by everything else it can reach

Context — Project Instructions Are the Kitchen's Rules

Concept	Claude Code	Codex	Gemini
Project instructions	`CLAUDE.md`	`AGENTS.md`	`GEMINI.md`
Purpose	Tell AI the project identity, norms, working style	Same idea	Same idea
Role today	Main focus	Reference	Reference

You're telling the AI: "This is our kitchen, this is how we do things."

> Write it once, and future conversations start from the same baseline

Context — What Usually Goes Into Project Instructions?

```
# Project Name

## Description
(...)

## Tech Stack
React + Vite, Express, MongoDB

## Coding Conventions
- Use functional components
- API responses use { data, error }

## Workflow
- Check RWD breakpoints before UI changes
- Run integration tests after API changes
```

It's not an encyclopedia — it defines how this kitchen operates

> Too little: AI guesses
> Too much: the important rules get drowned out

⚠️ ETH Zurich 2026:
AI-generated instruction files -> success rate actually **drops**
Human-written but overloaded -> barely helps

Context — How to Write Project Instructions That Actually Help

Do write	Don't write
Rules (constraints, what not to do)	Intro (what the project is, directory layout)
Team/repo-specific constraints	Common knowledge the model already has
Long-term processes	One-off tasks (make it a Skill or Hook)
Counter-examples "don't do this"	Empty platitudes like "stay elegant"

> Every line takes up global context
> Include only what would cause failure if missing — update after the same error happens twice

Context — The Prompt Is Actually Assembled

You're not writing just one CLAUDE.md — you're building composable modules

```
CLAUDE.md → role → policy → stack → skills → templates
═══════════════════════════   ═══════════════════════════
   Loaded every time (core)         Loaded on demand
```

- **Core modules**: identity, rules — travel with every conversation
- **Skills**: how to do it — loaded when the task has a fixed SOP
- **Templates**: structured information formats — task context, reports, proposals are all templates

Harness — How the Kitchen Stays Under Control

Mechanism	Restaurant metaphor	Description
Permissions / approval	Which actions need manager sign-off	High-risk ops confirm first
Hooks / auto checks	QA before the dish leaves	Run checks or flows at specific moments
Workflow constraints	Kitchen's standard flow	Read code, change, then verify

```
// .claude/settings.json — hook example
// In plain words: "every time AI edits a file, automatically run the linter"
{ "hooks": { "PostToolUse": [{
    "matcher": "Edit",
    "hooks": [{ "type": "command", "command": "pnpm lint --fix" }]
}]}}
```

Two Sides of Harness

Side	What it does	Restaurant metaphor
Automatic guardrails	hooks, permissions, checkpoints to prevent errors	QA checks before each plate leaves
Iterative tuning	Adjust Prompts & Skills based on outcomes	Review after each service, refine recipes and flow

Both together make it complete

- Only guardrails, no tuning –> rigid rules, new situations break things
- Only tuning, no guardrails –> someone has to babysit every time

> Runtime guardrails and after-the-fact tuning are separate, but both shape how stable the workflow becomes

Skills / MCP / Plugins — What Does Each Do?

Metaphor	Concept	In plain words
Ordering style	**Prompt**	How you give instructions
Kitchen rules	**CLAUDE.md / AGENTS.md**	Project context and ways of working
Standard operating flow	**Skills**	AI runs through steps for a given task
External interface	**MCP**	AI connects to outside tools or data
Equipment bundle	**Plugins**	Package common capabilities as reusable sets

> You're designing a kitchen that runs — not just buying more utensils

Skills — Turning Practices Into SOPs

A Skill can capture the flow for a common task, for example:

- Clarify the requirement first
- Then break down a plan
- Implement
- Finally verify and wrap up

```
fuzzy requirement -> brainstorming -> plan -> implementation -> verification
```

> A Skill is more than a prompt template — it's a repeatable workflow

MCP — Let the Kitchen Fetch Information Itself

MCP lets AI go get info or perform actions when needed, for example:

- Read issues or tickets
- Query database results
- See team discussion context
- Read files or docs

The key isn't "how many systems you hook up"

It's: **Can AI pick up the info it needs, at the right moment, on its own?**

> MCP is an external interface — not the external system itself

One-Line Recap: What Is Intermediate Really About?

Prompt

State the request clearly

Context

Let AI know the kitchen's rules and where to find information

Harness

Let AI work within the right boundaries and flow

> Intermediate isn't piling on more tools — it's starting to design **the AI's work environment**

Part 2: Live Demo

From 0 to 1 — let AI build a product prototype following a flow

```
Build a task management system with RWD
Needs a task list, status filter, new task form
Must work on desktop and mobile
```

How does the demo map to the three pillars?

- **Prompt / Context**: Set the role and rules first
- **Context –> Spec**: Use brainstorming to converge the requirement
- **Harness**: Use plan and feedback loops to advance and correct

Demo 1 — Set Up the Role / Skill System First

Create three Role Prompts in the project

- **Frontend**: focuses on UI structure, RWD, interaction, component decomposition
- **Backend**: focuses on API, data model, validation, error handling
- **QA**: focuses on test cases, edge conditions, risk flags

Each role gets a matching Skill

- **Frontend skill**: define the page structure, then breakpoints, then break down components
- **Backend skill**: define the schema, then the API, then validation
- **QA skill**: write a test plan, then cover edge cases and an E2E checklist

Demo 2 — Set Up the Constraints

Use `/init` to generate CLAUDE.md, then write the rules in

```
/init For this project, each task only carries one role and its Skill, with at most 3 Skills.
```

Scenario	Without constraints	With constraints
Role mixing	Front and back blended, style drifts	One role at a time, clear responsibility
Skill overload	Too many SOPs conflict	Up to 3, focused on the current task
Lost rules	Re-explain every conversation	Written in CLAUDE.md, applied automatically

Demo 3 — Brainstorm to Converge Requirements Into a Design Spec

Install Superpowers

Start from one fuzzy sentence

```
/superpowers:brainstorm I want to build a task management system with RWD
```

- Clarify step by step into an actionable spec
- Decide on architecture and tech stack

> The key isn't to write code immediately — it's to converge the requirement into a spec you can divide and sign off on

Demo 4 — From Plan to Step-by-Step Implementation

First, turn the spec into an executable plan

- Break the spec into **N tasks**
- Let AI implement each task in order
- Use build / test as the sign-off criteria

Execution order

1. Backend schema + API
2. Frontend components + pages
3. QA adds test cases and acceptance

> AI moves step by step along the plan, verifying after each step

Demo 5 — Correct in Real Time Based on Feedback

Boot up the environment to verify

- Have AI start backend and frontend automatically after changes
- Use the browser to exercise the app and check whether features actually work

Problem	What to patch	How to fix
Mobile task card layout is broken	Context (conventions)	Adjust flex / media query
New task returns 404	Prompt (routing detail)	Check routes and controller
"Completed" filter still shows	Harness (test	Confirm query and filter

Demo 6 — Use OpenSpec to Propose Requirement Changes

1. Install OpenSpec (`/opsx` is the slash command prefix it creates)
2. Enable it in the project with `openspec init`
3. `/opsx:explore`: raise the requirement
4. `/opsx:propose`: create the requirement doc and division of work
5. `/opsx:apply`: implement according to the spec
6. `/opsx:archive`: archive the requirement after sign-off

> OpenSpec isn't a document repository — it's a change ledger for requirements, design, implementation, and sign-off

Demo Recap

Highlight	What the audience should see
Role & Skill system	Define role, define SOP, then use rules to constrain AI
Superpowers	Fuzzy requirements can be gradually converged into an actionable spec
Plan-driven implementation	Break the spec into tasks, implement and sign off in order
Real-time feedback fixes	Judge whether to patch Prompt / Context / Harness
OpenSpec workflow	`explore -> propose -> apply -> archive` keeps every change in context
Feedback Loop	Use plan, verification, and feedback loops so the flow tightens over time

Beyond Skills, Another Useful Layer: Templates

Template	Use
`templates/task-context.md`	Single-task context (lightest)
`templates/change-context.md`	Single change (scope and impact included)
`templates/project-context.md`	Full project picture (most complete)
`templates/code-review.md`	Code Review report
`templates/design-proposal.md`	Design proposal

> Every template is "a structured information format"

Part 3: Exercise Time (20 min)

Apply today's methods to your own work environment

> Exercise 1 is required, Exercise 2 optional, Free practice is take-home extension

Exercise 1 — Design Your Team's Kitchen Rules

Your task

Draft a `CLAUDE.md` for your repo (or today's demo project)

Answer at least these four

1. What is this project? What tech stack?
2. What should AI know before touching the code?
3. What should AI verify after finishing?
4. What's off-limits?

> Writing a 60-point first draft beats trying to nail 100 points in one go

Exercise 2 — Add Harness to a Common Task

Pick a task your team does often, then define permissions and checkpoints

Question to answer	Example
What can AI do directly?	Read files, run lint
What should AI ask first?	Delete files, change DB schema
How do we verify afterward?	Run the test suite, check API responses

> Many crashes aren't because AI isn't smart — they're because there's no seatbelt

Free Practice (Take-Home)

Pick whichever is closest to your work

> **A)** Draft a `CLAUDE.md` for your team's repo
> **B)** Break a fuzzy requirement into Prompt / Context / Harness
> **C)** Pick a common task and design its Skill (from trigger conditions to acceptance criteria)

A reminder

Aim for something you can actually use when you get back

> Don't try to do a lot — ship one version you can put into practice

Part 4: Recap

Practical Reminder: Cost and Context Awareness

As mentioned earlier, context affects not only quality but also cost

These patterns tend to burn resources:

Behavior	Common effect	Suggestion
Marathon conversations	Context keeps growing	Start a fresh conversation to keep topics clean
Stuffing irrelevant info at once	Judgment quality drops	Only share what's actually relevant
Trial-and-error with no convergence	Eats time and budget	Step back and patch context or rules
Unnecessary tool chaining	Complexity rises	Solve the core problem first, then add tools

Key Takeaways

1. Prompt + Context + Harness — you need all three

Good results rarely come from one magic prompt; they come from patching all three layers together

2. `CLAUDE.md` / `AGENTS.md` are your team's shared kitchen rules

Write once, and future collaborations don't have to re-align the rules

3. Skills + MCP make AI part of the work environment, not just a chat tool

AI doesn't only answer — it follows a flow and fetches information when needed

> The core of intermediate isn't "using AI better" — it's "designing how AI

Take-Home Exercises

Exercise	How to do it
Draft a project instruction file	List the stack, conventions, verification flow, and off-limits
Run a four-round reinforcement experiment	Same task, patch one at a time: Prompt -> Context -> Harness -> Skill (compare results)
Turn a common flow into an SOP	Write out the steps in text first, then decide whether to make it a Skill
When results are unstable	Trace back: which layer — Prompt / Context / Harness — is undercooked?

Do one thing when you get back

> Add the most basic AI working rules to your repo

Next Stop: Advanced Session Preview

Today you learned — how to put AI into a stable work environment

The advanced session covers — how to compose multiple Agents into a continuously running workflow

Intermediate (today)	Advanced (next session)
Restaurant: how one kitchen runs	Factory: how multiple production lines coordinate
Single Agent + rules	Multi-Agent architecture + custom Skills
Prompt / Context / Harness	Agentic Workflow design

> The foundations laid today (roles, rules, checkpoints) all get reused in the advanced session

Q&A